The Journal of Open Source Software

mvs: Methods for high-dimensional multi-view learning

Wouter van Loon [1]

1 Department of Methodology and Statistics, Leiden University, the Netherlands

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 09 September 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

In biomedical science, a set of objects or persons can often be described by multiple distinct sets of *features* (also called *independent variables*) obtained from different data sources or modalities (Y. Li et al., 2018). These feature sets are said to provide different *views* of the objects or persons under consideration. Data sets consisting of multiple views are called *multi-view data* (Smilde et al., 2022; Sun et al., 2019; J. Zhao et al., 2017)[1]. Data from multiple views can be integrated using *multi-view stacking* (MVS) (Garcia-Ceja et al., 2018; R. Li et al., 2011; Van Loon et al., 2020). MVS is a form of supervised (machine) learning used to train multi-view classification or prediction models. MVS works by training a learning algorithm (the *base-learner*) on each view separately, estimating the predictive power of each view-specific model through cross-validation, and then using another learning algorithm (the *meta-learner*) to assign weights to the view-specific models based on their estimated predictions. MVS is a form of *ensemble learning*, dividing the large multi-view learning problem into smaller sub-problems, most of which can be solved in parallel, making it computationally attractive. Additionally, the number of features of the sub-problems is greatly reduced compared to the full multi-view learning problem. This makes MVS especially useful when the total number of features is larger than the number of observations (*high-dimensional* data). MVS can still be applied even if the sub-problems are themselves high-dimensional by adding suitable *penalty terms* to the learning algorithms (Van Loon et al., 2020). Furthermore, MVS can automatically select the views which are most important for prediction (Van Loon et al., 2020). The R package `mvs` makes fitting MVS models, including such penalty terms, easily and openly accessible. `mvs` allows fitting stacked models with any number of levels, with different penalty terms, different outcome distributions, and provides several options for missing data handling.

Statement of need

The multi-view stacking methodology was introduced by Li et al. (2011), and recently popularized by Garcia-Ceja et al. (2018), but no software packages implementing the methodology were made publicly available. We further investigated and extended the methodology, and found that it has many favorable properties (Van Loon et al., 2020, 2022, 2024; Van Loon, Fokkema, De Vos, et al., 2024). We now present the R package `mvs` which makes fitting MVS models easily accessible to the scientific community. A small number of packages for multi-view learning were available on the Comprehensive R Archive Network (CRAN) at the time of the first release of `mvs`: `Spectrum` (John & Watson, 2020), `LUCIDus` (Y. Zhao, 2022) and `multiview` (Ding et al., 2023), but none of these packages included multi-view stacking. Development versions of `mvs` have been used, among other applications, to analyze multi-view gene expression (Van Loon et al., 2020, 2024) and multi-view magnetic resonance imaging (MRI) data (Van Loon et al., 2022; Van Loon, Fokkema, De Vos, et al., 2024).

[1] Depending on the research area, multi-view data is sometimes called multi-block, multi-set, multi-group, or multi-table data (Smilde et al., 2022).

Functionality

mvs is available on CRAN and GitLab. The repository includes documentation with examples of usage.

Model fitting

The primary application of mvs is to fit multi-view stacking (MVS) models. The implementation of MVS is based on an extension of the *Stacked Penalized Logistic Regression* (StaPLR) algorithm (Van Loon et al., 2020). mvs features two main functions for fitting MVS models:

- StaPLR is used to fit penalized and stacked penalized regression models with up to two levels. The minimum required input is the total feature matrix (x), the outcome variable (y), and a vector denoting to which view each feature corresponds (view_index).
- MVS is used to fit multi-view stacking models with two or more levels, which are appropriate when the data have a hierarchical multi-view structure, that is, the features are nested in views, which are themselves nested in larger views, and so on (Van Loon et al., 2022). The minimum required input is the total feature matrix (x), the outcome variable (y), and a matrix of dimensions [*number of features* $\times$ (*number of levels* -1)] denoting to which view each feature corresponds at each level (views).

For more technical arguments see the documentation included with mvs. The individual sub-problems are optimized using coordinate descent via the R package glmnet (Friedman et al., 2010). Users of glmnet will feel right at home since mvs uses a very similar syntax.

Parallelization

One of the main advantages of MVS is that, at each level of the hierarchy, *all sub-problems are independent*. This means that these sub-problems can be calculated in parallel. mvs supports parallel computation through foreach (Microsoft & Weston, 2022), assuming a parallel back-end is registered (for more information about registering a parallel back-end in R see Weston & Calaway, 2015). Parallel computation can be enabled using the function argument parallel.

Model generalizations

- Although originally developed for binary outcome variables, mvs can be used to model outcome variables with different distributions. Binomial, Gaussian and Poisson distributions are currently supported through the function argument family.
- As of version 2.0.0, mvs supports the use of model relaxation (as used in, e.g., the relaxed lasso (Hastie et al., 2017)). Model relaxation can be enabled for the entire hierarchy, or only for specific levels, through the function argument relax.
- As of version 2.0.0, mvs supports the use of adaptive weights (as used in, e.g., the adaptive lasso (Zou, 2006)). Adaptive weights can be enabled for the entire hierarchy, or only for specific levels, through the function argument adaptive. Adaptive weights are initialized using ridge regression as described in Van Loon, Fokkema, Szabo, et al. (2024).

View importance

In a two-level StaPLR model, the meta-level regression coefficient of each view can be used as a measure of that view's importance, since these regression coefficients are effectively on the same scale (Van Loon et al., 2020). In hierarchical StaPLR/MVS models with more than two levels this does not necessarily apply, since these coefficients may correspond to different sub-models at different levels of the hierarchy. The *minority report measure* (MRM) was developed to quantify the importance of a view at any level of the hierarchy (Van Loon et al.,

DRAFT

2022). The MRM quantifies how much the prediction of the complete stacked model changes as the view-specific prediction of view i changes from a (default value 0) to b (default value 1), while the other predictions are kept constant (the recommended value being the mean of the outcome variable) (Van Loon et al., 2022). As of version 2.0.0, the MRM can be calculated using `MRM`.

Handling missing data

In practice, it is likely that not all views are measured for all observations. When a view is missing for some observations, typical approaches are:

1. Remove any observations with at least one missing value (*list-wise deletion*)
2. Replace any missing values with values calculated from the observed data (*imputation*)

The first approach is not recommended since it is very wasteful; often more values are removed through list-wise deletion than were initially missing. The second approach is preferable, but often unrealistic. For example, if the missing view is an MRI scan, this means having to impute millions of values, often from high-dimensional observed data, which is computationally infeasible for all but the simplest imputation algorithms (Van Loon, Fokkema, De Vos, et al., 2024). As of version 2.0.0, `mvs` thefore supports a third option: *meta-level imputation*. Instead of imputing the raw data, meta-level imputation uses the complete observations for each view to generate cross-validated predictions, and performs imputation in the reduced space. This is much faster than traditional imputation and leads to comparable performance (Van Loon, Fokkema, De Vos, et al., 2024). It allows the use of state-of-the-art imputation algorithms which would otherwise be too computationally intensive (Van Loon, Fokkema, De Vos, et al., 2024). The following imputation methods are currently supported:

- `mean` performs meta-level (unconditional) mean imputation.
- `mice` performs meta-level predictive mean matching. It requires the R package `mice` (van Buuren & Groothuis-Oudshoorn, 2011).
- `missForest` performs meta-level missForest imputation. It requires the R package `missForest` (Stekhoven & Bühlmann, 2012).

Additionally, `mvs` includes the option to 'pass' the missing values through to the meta-level without imputing them, allowing the user to use a different imputation scheme of their choice. Options for missing data handling can be specified directly through model fitting using the arguments `na.action` and `na.arguments`. For more details about meta-level imputation see Van Loon, Fokkema, De Vos, et al. (2024).

Acknowledgements

I acknowledge the contribution of Marjolein Fokkema, who helped prepare `mvs` for submission to CRAN and worked on the implementation of model relaxation. I acknowledge the support of Mark de Rooij and Botond Szabo during the earlier stages of this project.

References

Ding, D. Y., Tibshirani, R. J., Narasimhan, B., Hastie, T., Tay, K., & Yang, J. (2023). *Multiview: Cooperative learning for multi-view analysis*. https://doi.org/10.32614/CRAN.package.multiview

Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, *33*(1), 1–22. https://doi.org/10.18637/jss.v033.i01

Garcia-Ceja, E., Galván-Tejada, C. E., & Brena, R. (2018). Multi-view stacking for activity recognition with sound and accelerometer data. *Information Fusion*, *40*, 45–56. https://doi.org/10.1016/j.inffus.2017.06.004

Hastie, T., Tibshirani, R., & Tibshirani, R. J. (2017). Extended comparisons of best subset selection, forward stepwise selection, and the lasso. *arXiv Preprint arXiv:1707.08692*. https://doi.org/10.1214/19-sts733

John, C. R., & Watson, D. (2020). *Spectrum: Fast adaptive spectral clustering for single and multi-view data*. https://doi.org/10.32614/CRAN.package.Spectrum

Li, R., Hapfelmeier, A., Schmidt, J., Perneczky, R., Drzezga, A., Kurz, A., & Kramer, S. (2011). A case study of stacked multi-view learning in dementia research. *13th Conference on Artificial Intelligence in Medicine*, 60–69.

Li, Y., Wu, F.-X., & Ngom, A. (2018). A review on machine learning principles for multi-view biological data integration. *Briefings in Bioinformatics*, *19*(2), 325–340. https://doi.org/10.1093/bib/bbw113

Microsoft, & Weston, S. (2022). *Foreach: Provides foreach looping construct*. https://doi.org/10.32614/CRAN.package.foreach

Smilde, A. K., Næs, T., & Liland, K. H. (2022). *Multiblock data fusion in statistics and machine learning: Applications in the natural and life sciences*. John Wiley & Sons.

Stekhoven, D. J., & Bühlmann, P. (2012). MissForest - non-parametric missing value imputation for mixed-type data. *Bioinformatics*, *28*(1), 112–118. https://doi.org/10.1093/bioinformatics/btr597

Sun, S., Mao, L., Dong, Z., & Wu, L. (2019). *Multiview machine learning*. Springer-Verlag. https://doi.org/10.1007/978-981-13-3029-2

van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, *45*(3), 1–67. https://doi.org/10.18637/jss.v045.i03

Van Loon, W., De Vos, F., Fokkema, M., Szabo, B., Koini, M., Schmidt, R., & De Rooij, M. (2022). Analyzing hierarchical multi-view MRI data with StaPLR: An application to Alzheimer's disease classification. *Frontiers in Neuroscience*, *16*(83063). https://doi.org/10.3389/fnins.2022.830630

Van Loon, W., Fokkema, M., De Vos, F., Koini, M., Schmidt, R., & De Rooij, M. (2024). Imputation of missing values in multi-view data. *Information Fusion*, *111*, 102524. https://doi.org/10.1016/j.inffus.2024.102524

Van Loon, W., Fokkema, M., Szabo, B., & De Rooij, M. (2020). Stacked penalized logistic regression for selecting views in multi-view learning. *Information Fusion*, *61*, 113–123. https://doi.org/10.1016/j.inffus.2020.03.007

Van Loon, W., Fokkema, M., Szabo, B., & De Rooij, M. (2024). View selection in multi-view stacking: Choosing the meta-learner. *Advances in Data Analysis and Classification*. https://doi.org/10.1007/s11634-024-00587-5

Weston, S., & Calaway, R. (2015). *Getting started with doParallel and foreach*. https://cran.r-project.org/web/packages/doParallel/vignettes/gettingstartedParallel.pdf

Zhao, J., Xie, X., Xu, X., & Sun, S. (2017). Multi-view learning overview: Recent progress and new challenges. *Information Fusion*, *38*, 43–54. https://doi.org/10.1016/j.inffus.2017.02.007

Zhao, Y. (2022). *LUCIDus: An R package to implement the LUCID model*. https://doi.org/10.32614/CRAN.package.LUCIDus

DRAFT

Zou, H. (2006). The adaptive lasso and its oracle properties. *Journal of the American Statistical Association*, *101*(476), 1418–1429. https://doi.org/10.1198/016214506000000735

DRAFT